



|                                                                                                                                      |  |
|--------------------------------------------------------------------------------------------------------------------------------------|--|
|                                                   |  |
|  Malta<br>Further & Higher<br>Education Authority |  |

**This cover sheet must be completed and added to the front of every assignment**

|                                  |                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------|
| <b>Learner Name and Surname:</b> | Imran Hossain Chowdhury                                                                    |
| <b>Learner Registration No.</b>  | 11248                                                                                      |
| <b>Study Centre Name</b>         | <b>Learn Key Institute</b>                                                                 |
| <b>Qualification Title</b>       | Undergraduate Diploma in Software Design MQF (Lv.5) — Group A                              |
| <b>Unit Reference No.</b>        | ICT-UDSDU2-LKI/MQF (Lv.5)                                                                  |
| <b>Unit Title</b>                | Unit 3: Data Structures and Algorithms in Software                                         |
| <b>Assignment Title</b>          | Inserting a New Cell at the End of a Linked List                                           |
| <b>Submission Date</b>           |                                                                                            |
| <b>Word Count</b>                | 4,339 words (excluding the front page, contents, figures, code, references and appendices) |

**Declaration of authenticity:**

- 1. I declare that the attached submission is my own original work. No significant part of it has been submitted for any other assignment and I have acknowledged in my notes and bibliography all written and electronic sources used.**
- 2. I acknowledge that my assignment will be subject to electronic scrutiny for academic honesty.**
- 3. I understand that failure to meet these guidelines may instigate the centre's malpractice procedures and risk failure of the unit and / or qualification.**

|                                                |                                              |
|------------------------------------------------|----------------------------------------------|
| <hr/> <i>Learner signature</i><br><i>Date:</i> | <hr/> <i>Tutor signature</i><br><i>Date:</i> |
|------------------------------------------------|----------------------------------------------|

Note:

Assignments must be submitted in typed PDF format only; handwritten assignments are not accepted.

# Table of Contents

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>1 Introduction.....</b>                                      | <b>4</b>  |
| <b>2 Creating the Linked List.....</b>                          | <b>5</b>  |
| 2.1 The cell structure.....                                     | 5         |
| 2.2 The list created.....                                       | 5         |
| 2.3 The initialisation algorithm.....                           | 6         |
| 2.4 Validation and error handling.....                          | 6         |
| 2.5 The list in memory.....                                     | 6         |
| <b>3 The Insertion Algorithm: Processes and Techniques.....</b> | <b>8</b>  |
| 3.1 The algorithm.....                                          | 8         |
| 3.2 Process 1: validating New (lines 1–2).....                  | 8         |
| 3.3 Process 2: handling an empty list (lines 3–6).....          | 8         |
| 3.4 Process 3: finding the last cell (lines 7–13).....          | 8         |
| 3.5 Process 4: re-linking (lines 14–16).....                    | 9         |
| 3.6 Justification of the approach.....                          | 10        |
| <b>4 Documentation of Components.....</b>                       | <b>12</b> |
| 4.1 Where the data is stored.....                               | 12        |
| 4.2 Variables and pointers.....                                 | 12        |
| 4.3 Instructions and commands.....                              | 13        |
| 4.4 How the components work together.....                       | 14        |
| <b>5 Logic Explanation.....</b>                                 | <b>16</b> |
| 5.1 How the last cell is identified.....                        | 16        |
| 5.2 Why the loop is correct.....                                | 16        |
| 5.3 The transition from $\Lambda$ to New.....                   | 16        |
| 5.4 Edge cases.....                                             | 17        |
| 5.5 Preserving list integrity.....                              | 18        |
| 5.6 Tracing the algorithm.....                                  | 18        |
| <b>6 Flowchart and Explanation.....</b>                         | <b>20</b> |
| 6.1 The flowchart.....                                          | 20        |
| 6.2 Explanation of each step.....                               | 21        |
| 6.3 From flowchart to code.....                                 | 21        |
| <b>7 Implementation and Testing.....</b>                        | <b>23</b> |
| <b>8 Conclusion and Self-Assessment.....</b>                    | <b>24</b> |

|                                        |           |
|----------------------------------------|-----------|
| <b>References.....</b>                 | <b>25</b> |
| <b>Appendix A: Source Code.....</b>    | <b>26</b> |
| A.1 linked_list.hpp.....               | 26        |
| A.2 linked_list.cpp.....               | 26        |
| A.3 main.cpp.....                      | 28        |
| A.4 test_linked_list.cpp.....          | 29        |
| <b>Appendix B: Program Output.....</b> | <b>34</b> |
| B.1 Demonstration program.....         | 34        |
| B.2 Test program.....                  | 34        |

# 1 Introduction

A linked list stores a sequence of values in separate cells that are joined by memory addresses rather than by their position in memory. Each cell has two fields: Info, which holds the data, and Link, which holds the address of the next cell. A variable called Head holds the address of the first cell, and the Link of the last cell holds a special null value, written  $\Lambda$ , meaning that no cell follows (Knuth, 1997).

This report addresses the task set in the assignment brief. Given a list pointed to by Head, and a separate cell pointed to by New whose Info field holds the value  $X(n+1)$ , the algorithm must change the Link field of the last cell of the list so that it points to New. The effect is to insert a new cell at the end of the list.

The work was implemented in C++17. The language was chosen because its raw pointers make every step of the algorithm visible: Head, New and Temp are genuine memory addresses, `nullptr` plays the role of  $\Lambda$ , and cells are created and released explicitly with `new` and `delete` (Stroustrup, 2013).

Sections 2 to 6 follow the five requirements of the brief in order: creating the list, the processes and techniques, the documentation of components, the logic and the flowchart. Section 7 summarises the testing, Section 8 concludes, and the appendices give the full source code and program output.

In the pseudocode, `Info(P)` and `Link(P)` mean the Info and Link fields of the cell whose address is held in P, and  $\leftarrow$  means assignment. In C++ these are written `P->Info`, `P->Link` and `=`.

## 2 Creating the Linked List

### 2.1 The cell structure

Each cell is defined as a C++ structure with two members:

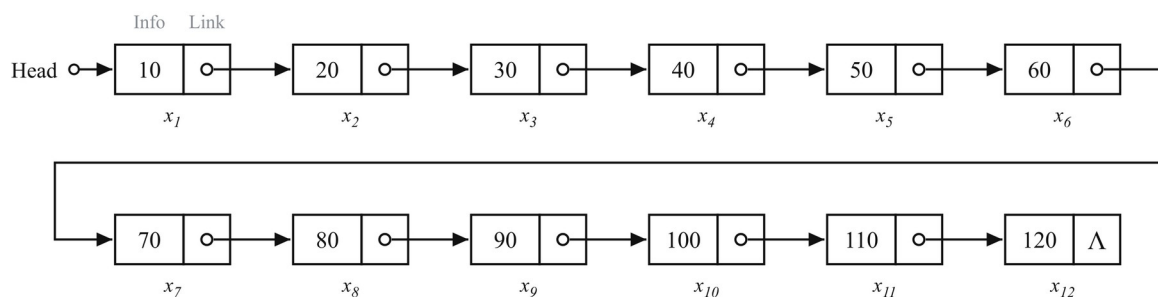
```
struct Cell {  
    int    Info;  
    Cell*  Link;  
};
```

Info is an integer holding the value of the cell. Link is a pointer to another Cell, which means it holds that cell's memory address. Because the structure refers to its own type through Link, cells can be chained together. On the 64-bit system used for this work, one Cell occupies 16 bytes: 4 bytes for Info, 4 bytes of padding added by the compiler so that Link starts on an 8-byte boundary, and 8 bytes for Link. These sizes were measured with `sizeof` and `offsetof` rather than assumed (Appendix B).

The list as a whole is reached through one more pointer, declared as `Cell* Head`. Head is not a cell: it is an 8-byte variable holding the address of the first cell, or  $\Lambda$  when the list is empty.

### 2.2 The list created

The brief requires a list of between 10 and 16 cells. Twelve cells were used, holding the values  $x_1 = 10$ ,  $x_2 = 20$  and so on up to  $x_{12} = 120$ , as shown in Figure 1. Twelve was chosen deliberately: after the new cell is inserted the list has thirteen cells, so it stays within the permitted range both before and after the insertion.



**Figure 1:** The twelve-cell list built by `createList`. Head points to  $x_1$ , each Link points to the next cell, and the Link of  $x_{12}$  is  $\Lambda$ .

## 2.3 The initialisation algorithm

The list is built by the function `createList`, which receives the twelve values in order and returns the address of the first cell:

```
Algorithm CreateList(values)
1  Head  $\leftarrow \Lambda$ ; Tail  $\leftarrow \Lambda$ 
2  for each value  $v$  in values
3      P  $\leftarrow$  address of a new cell
4      Info(P)  $\leftarrow v$ ; Link(P)  $\leftarrow \Lambda$ 
5      if Head =  $\Lambda$  then Head  $\leftarrow$  P
6      else Link(Tail)  $\leftarrow$  P
7      Tail  $\leftarrow$  P
8  return Head
```

Every new cell is created with its Link already set to  $\Lambda$  (line 4). When the first cell is created, Head is set to point to it (line 5). Each later cell is attached by writing its address into the Link field of the previous last cell, which is tracked by the pointer Tail (line 6). Because a Link stays  $\Lambda$  until the next cell is attached, the list has a valid end at every stage of its construction.

Tail keeps construction to  $O(n)$  time. Without it, adding each cell would require a walk from Head to the end of the list, making the construction  $O(n^2)$  (Cormen et al., 2022).

## 2.4 Validation and error handling

Two safeguards make the initialisation error-free. First, `createList` checks the number of values before allocating any memory and throws `std::invalid_argument` if it is outside the range 10 to 16, so a list of the wrong size can never be created. Second, if memory allocation fails part-way through, `new` throws `std::bad_alloc`. `createList` catches the exception, releases every cell already built by calling `deleteList`, and then passes the exception on. The program therefore never leaves a partly built list or unreleased memory behind.

## 2.5 The list in memory

Table 1 shows the list as it exists in memory, taken from the output of the program. Head holds the address of `x1`, each Link holds the address of the following cell, and the Link of `x12` is  $\Lambda$ .

**Table 1:** *The twelve-cell list in memory*

| Cell | Address     | Info | Link        |
|------|-------------|------|-------------|
| x1   | 0x1036f5b70 | 10   | 0x1036f5900 |
| x2   | 0x1036f5900 | 20   | 0x1036f5910 |
| x3   | 0x1036f5910 | 30   | 0x1036f5960 |
| x4   | 0x1036f5960 | 40   | 0x1036f5970 |
| x5   | 0x1036f5970 | 50   | 0x1036f5980 |
| x6   | 0x1036f5980 | 60   | 0x1036f5990 |
| x7   | 0x1036f5990 | 70   | 0x1036f59a0 |
| x8   | 0x1036f59a0 | 80   | 0x1036f59b0 |
| x9   | 0x1036f59b0 | 90   | 0x1036f59c0 |
| x10  | 0x1036f59c0 | 100  | 0x1036f59d0 |
| x11  | 0x1036f59d0 | 110  | 0x1036f59e0 |
| x12  | 0x1036f59e0 | 120  | $\Lambda$   |

The addresses are not consecutive. The cell x1 lies in a different region of memory from x2 to x12, and x4 is 80 bytes after x3 rather than 16. This shows the defining property of a linked list: unlike the elements of an array, its cells do not need to be stored next to one another, because each cell records where the next one is (Weiss, 2014). The exact addresses change from one run to the next, because the operating system varies where memory is allocated.

## 3 The Insertion Algorithm: Processes and Techniques

### 3.1 The algorithm

The complete algorithm, written in the notation of the brief, is:

```
Algorithm InsertAtEnd(Head, New)
1  if New =  $\Lambda$  then
2      return false
3  if Head =  $\Lambda$  then
4      Link(New)  $\leftarrow \Lambda$ 
5      Head  $\leftarrow$  New
6      return true
7  Temp  $\leftarrow$  Head
8  loop
9      if Temp = New then
10         return false
11     if Link(Temp) =  $\Lambda$  then
12         exit loop
13     Temp  $\leftarrow$  Link(Temp)
14 Link(New)  $\leftarrow \Lambda$ 
15 Link(Temp)  $\leftarrow$  New
16 return true
```

The algorithm returns true if the cell was inserted and false if it was refused, so that the calling code knows whether New now belongs to the list. It consists of four processes, described below and illustrated in Figure 2.

### 3.2 Process 1: validating New (lines 1–2)

The algorithm first checks that there is a cell to insert. If New is  $\Lambda$  it holds no address, and any attempt to write to Link(New) would access invalid memory. The algorithm returns false and leaves the list unchanged.

### 3.3 Process 2: handling an empty list (lines 3–6)

If Head is  $\Lambda$ , the list has no cells, so there is no last cell whose Link can be changed. In this case New becomes the whole list: its Link is set to  $\Lambda$  and Head is set to point to it. This is the only situation in which Head itself changes, which is why the C++ function receives Head by reference (Section 4.2).

### 3.4 Process 3: finding the last cell (lines 7–13)

The last cell cannot be reached directly, because the list only records the address of its first cell. It has to be found by traversal. A pointer called Temp starts at the first cell (line 7). On

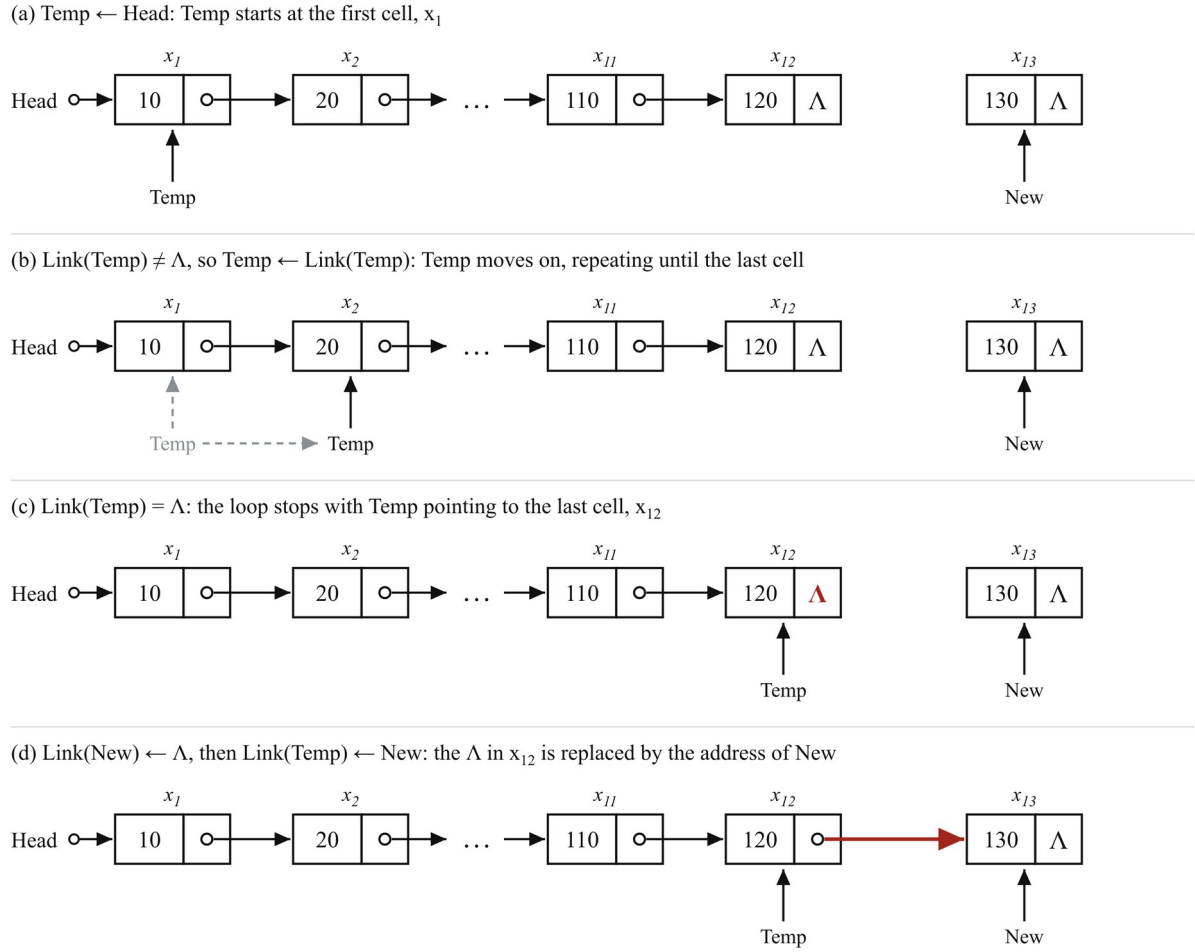
each pass through the loop the algorithm tests whether  $\text{Link}(\text{Temp})$  is  $\Lambda$  (line 11). If it is not, Temp is moved forward by copying into it the address held in its own cell's Link field (line 13). When  $\text{Link}(\text{Temp})$  is  $\Lambda$ , Temp is on the last cell and the loop ends (line 12).

This technique is a linear traversal, a form of sequential search: each cell is visited once, in order, until one that satisfies the condition is found (Levitin, 2012). Repeatedly replacing a pointer with the Link of the cell it points to is known as following, or chasing, pointers. On the twelve-cell list the loop makes twelve passes and moves Temp eleven times.

The loop also makes a second test (line 9): whether Temp has reached New itself. This detects the case where New is already part of the list, which is explained in Section 5.5.

### **3.5 Process 4: re-linking (lines 14–16)**

Once Temp points to the last cell, two assignments complete the insertion. Line 14 sets  $\text{Link}(\text{New})$  to  $\Lambda$ , so that New is a valid last cell. Line 15 then overwrites the  $\Lambda$  in  $\text{Link}(\text{Temp})$  with the address of New. At that moment New becomes reachable from Head and the list has grown by one cell. No data is moved or copied: the whole insertion is achieved by changing a single address, which is the main advantage of linked allocation over sequential storage (Knuth, 1997). The order of the two assignments matters, as Section 5.3 explains.



**Figure 2:** The four stages of *insertAtEnd* on the twelve-cell list: (a) Temp starts at Head; (b) Temp moves from cell to cell; (c) the loop stops at  $x_{12}$ , whose Link is  $\Lambda$ ; (d) the  $\Lambda$  in  $x_{12}$  is replaced by the address of New.

### 3.6 Justification of the approach

The chosen algorithm is iterative, uses one extra pointer and works on the list exactly as the brief defines it, with only a Head pointer. Table 2 compares it with the alternatives that were considered.

**Table 2:** Alternative techniques considered

| Technique                         | Time   | Reason it was not chosen                                                                                                                                                                             |
|-----------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Keep a Tail pointer with the list | $O(1)$ | Fastest, but the list in the brief records only Head. Every operation that changes the list would also have to keep Tail correct, adding a second pointer that could fall out of step with the list. |

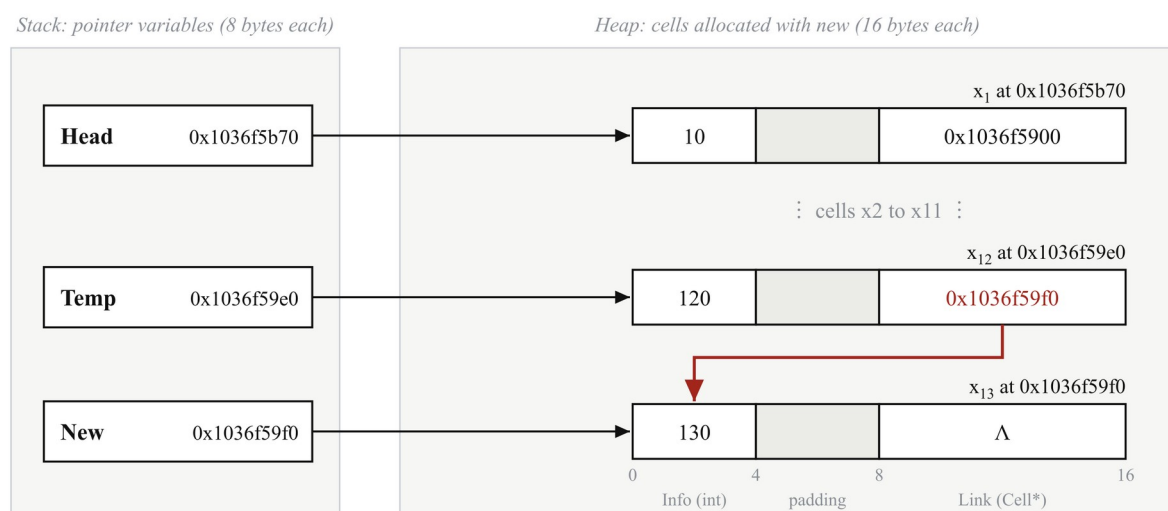
| Technique                                   | Time   | Reason it was not chosen                                                                                                                                       |
|---------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Recursive traversal                         | $O(n)$ | Each cell adds a function call to the stack, so memory use grows with the list and a very long list could overflow the stack. The loop needs only one pointer. |
| Dummy head (sentinel) cell                  | $O(n)$ | Removes the empty-list special case, but changes the structure given in the brief, where Head points to the first data cell (Cormen et al., 2022).             |
| <code>std::list</code> from the C++ library | $O(1)$ | Correct and efficient, but it hides the pointer manipulation that the task asks to be shown and explained.                                                     |
| <code>std::unique_ptr</code> for Link       | $O(n)$ | Releases memory automatically, but each re-link becomes a transfer of ownership, and destroying a long chain releases its cells recursively.                   |

The iterative traversal runs in  $O(n)$  time, because each of the  $n$  cells is visited once, and needs  $O(1)$  extra space, because only Temp is added however long the list becomes (Sedgewick and Wayne, 2011). For a list of 10 to 16 cells the difference between  $O(n)$  and  $O(1)$  is negligible, while keeping to the structure in the brief makes the algorithm simpler to verify.

## 4 Documentation of Components

### 4.1 Where the data is stored

The program uses two regions of memory, shown in Figure 3. The pointer variables `Head`, `New` and `Temp` are local variables with automatic storage, held on the stack. Each occupies 8 bytes and contains nothing but an address. The cells are created with `new` and have dynamic storage on the heap, also called the free store, where they remain until they are released with `delete` (Stroustrup, 2013). A pointer on the stack gives access to a cell on the heap, and the `Link` inside that cell gives access to the next one.



**Figure 3:** Memory during the final step of the insertion, using addresses from the program run. The pointers are on the stack; the cells are on the heap. The `Link` of `x12` now holds `0x1036f59f0`, the address of `New`.

### 4.2 Variables and pointers

Table 3 documents every variable and pointer used, the type it has in C++, where it is stored in memory and the part it plays in the algorithm.

**Table 3:** Variables and pointers

| Name | C++ type                                                                             | Stored in | Role                                                                                                                                                                                                     |
|------|--------------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Head | <code>Cell*</code> , received by <code>insertAtEnd</code> as <code>Cell*&amp;</code> | Stack     | Holds the address of the first cell, or $\Lambda$ when the list is empty. <code>insertAtEnd</code> receives it by reference so that it can change the caller's <code>Head</code> when the list is empty. |

| Name                                                   | C++ type                   | Stored in                   | Role                                                                                                                              |
|--------------------------------------------------------|----------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| New                                                    | Cell*                      | Stack                       | Holds the address of the cell to be inserted. The algorithm reads and writes that cell through New but never changes New itself.  |
| Temp                                                   | Cell*                      | Stack                       | The traversal pointer. It starts with the value of Head and moves one cell at a time until it holds the address of the last cell. |
| Tail                                                   | Cell*                      | Stack                       | Used only in createList, to hold the address of the most recently created cell.                                                   |
| P, called <code>cell</code> in C++                     | Cell*                      | Stack                       | In createList, holds the address of the cell just allocated until it is linked in.                                                |
| v, called <code>value</code> in C++; <code>info</code> | int                        | Stack                       | The value for the cell being created. createList passes it to createCell, which stores it in Info.                                |
| next                                                   | Cell*                      | Stack                       | In deleteList, saves Link(Head) before that cell is deleted, so the rest of the list stays reachable.                             |
| Info                                                   | int, 4 bytes               | Heap, offset 0 in each cell | The data value of a cell. The insertion never reads or changes it.                                                                |
| Link                                                   | Cell*, 8 bytes             | Heap, offset 8 in each cell | The address of the next cell, or $\Lambda$ . It is the only field the insertion changes.                                          |
| $\Lambda$                                              | nullptr                    | Not stored separately       | The null pointer, meaning "no cell". It marks the end of a list and an empty list.                                                |
| MIN_CELLS, MAX_CELLS                                   | constexpr<br>std::size_t   | Fixed when compiled         | The limits 10 and 16 from the brief, checked by createList.                                                                       |
| values                                                 | const<br>std::vector<int>& | Stack, as a reference       | The Info values passed to createList by reference, so they are not copied.                                                        |

### 4.3 Instructions and commands

Table 4 documents every instruction and command used in the implementation, with its meaning and its effect on memory.

**Table 4: Instructions and commands**

| Instruction                                   | Meaning                                            | Effect on memory                                                                                                |
|-----------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>struct Cell<br/>{ ... };</code>         | Declares the cell type                             | Defines the 16-byte layout of every cell but allocates nothing                                                  |
| <code>new<br/>Cell{info,<br/>nullptr}</code>  | Creates a cell                                     | Allocates 16 bytes on the heap, stores Info and a null Link, and returns the address                            |
| <code>delete P</code>                         | Destroys the cell at address P                     | Returns the 16 bytes to the heap. P still holds the old address, so it must not be used afterwards              |
| <code>P-&gt;Info,<br/>P-&gt;Link</code>       | Access to a field through a pointer                | Reads or writes a field of the cell at address P; written Info(P) and Link(P) in the pseudocode                 |
| <code>=</code>                                | Assignment, written $\leftarrow$ in the pseudocode | Copies a value or an address into a variable or field                                                           |
| <code>==, !=</code>                           | Comparison                                         | Compares two addresses without changing memory                                                                  |
| <code>if, else</code>                         | Selection                                          | Chooses which statements run; <code>createList</code> uses <code>else</code> to link every cell after the first |
| <code>while (true)<br/>with break</code>      | A loop whose exit test is inside it                | Repeats the traversal; <code>break</code> leaves the loop once the last cell is found                           |
| <code>while<br/>(Head !=<br/>nullptr)</code>  | A loop tested before each pass                     | <code>deleteList</code> repeats it until every cell has been released                                           |
| <code>for (int<br/>value :<br/>values)</code> | Range-based loop                                   | Visits each value once, so <code>createList</code> allocates exactly one cell per value                         |
| <code>sizeof,<br/>offsetof</code>             | Size and position operators, fixed when compiled   | Measure the 16-byte cell and the offsets of Info (0) and Link (8)                                               |
| <code>return false,<br/>return true</code>    | Ends the function with a result                    | Reports whether New was inserted                                                                                |
| <code>Cell*&amp;</code>                       | Reference to a pointer                             | Lets <code>insertAtEnd</code> assign directly to the caller's Head variable                                     |
| <code>throw, try,<br/>catch</code>            | Exception handling                                 | Used by <code>createList</code> to reject a wrong size and to release cells if allocation fails                 |

## 4.4 How the components work together

The insertion writes to memory in only two places: the Link field of New, which is set to  $\Lambda$ , and the Link field of the last cell, which changes from  $\Lambda$  to the address of New. In the program run shown in Figure 3, Temp holds 0x1036f59e0, the address of x12, and the Link

field of `x12` changes from `Λ` to `0x1036f59f0`, the address held in `New`. `insertAtEnd` itself creates, moves and deletes no cells. Creating `New` is the responsibility of the calling code, and releasing every cell afterwards is the responsibility of `deleteList`, which saves the address in each cell's `Link` before deleting that cell, so that the rest of the list is not lost.

## 5 Logic Explanation

### 5.1 How the last cell is identified

The last cell is the only cell whose Link field is  $\Lambda$ , because every other cell's Link holds the address of a following cell. The algorithm therefore does not need to know how many cells the list contains: it follows the Links until it reaches the one that is  $\Lambda$ . This is why correct null termination during creation (Section 2.3) is essential. A list whose last Link held a stray address instead of  $\Lambda$  would have no recognisable end.

### 5.2 Why the loop is correct

The correctness of the traversal can be shown with a loop invariant, a statement that is true every time execution reaches the start of the loop (Hoare, 1969):

Temp points to a cell in the list, and every cell before Temp has been checked and is neither New nor the last cell.

The invariant holds on the first pass, because Temp equals Head and no cells come before it. A pass that does not exit confirms that the current cell is neither New nor the last cell and then moves Temp to the next cell, so the invariant still holds at the start of the following pass. The loop must end, because a correctly terminated list has a finite number of cells and Temp advances by exactly one cell per pass. When the loop ends at line 12, Link(Temp) is  $\Lambda$ , so by the invariant Temp points to the last cell and New is not already in the list.

### 5.3 The transition from $\Lambda$ to New

The two assignments that complete the insertion are carried out in a fixed order:

1. Link(New)  $\leftarrow \Lambda$
2. Link(Temp)  $\leftarrow$  New

This order preserves the integrity of the list at every step. The first assignment makes New a correctly terminated cell while it is still outside the list. The second assignment is the single step that joins it to the list: the  $\Lambda$  that ended the list is replaced by the address of New, so the old last cell now points to the new last cell, and the new last cell points to nothing (Figure 2d).

If the order were reversed, New would become reachable from Head before its Link had been cleared. If New arrived pointing at another cell, that cell and every cell after it would briefly

become part of the list. Clearing  $\text{Link}(\text{New})$  first means there is no moment at which the list contains a cell that does not belong to it.

## 5.4 Edge cases

Table 5 lists the unusual inputs the algorithm was designed to handle.

**Table 5:** Edge cases and how they are handled

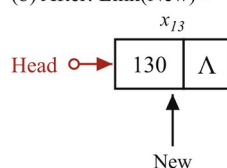
| Case                                | What the algorithm does                                                                                                             | Result                                           |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Empty list: $\text{Head} = \Lambda$ | There is no last cell to change, so $\text{Link}(\text{New}) \leftarrow \Lambda$ and $\text{Head} \leftarrow \text{New}$ (Figure 4) | New becomes the first and only cell              |
| List of one cell                    | $\text{Link}(\text{Temp})$ is already $\Lambda$ on the first pass, so Temp stays on the first cell                                  | $\text{Link}(\text{Head}) \leftarrow \text{New}$ |
| $\text{New} = \Lambda$              | Rejected before any memory is read or written                                                                                       | Returns false; list unchanged                    |
| New arrives with a non-null Link    | $\text{Link}(\text{New}) \leftarrow \Lambda$ clears it before New is joined                                                         | Only New is added                                |
| New is already in the list          | The loop reaches a cell where $\text{Temp} = \text{New}$                                                                            | Returns false; list unchanged                    |
| Repeated insertions                 | Each insertion finds the current last cell                                                                                          | List stays valid; tested from 10 to 16 cells     |

The empty list needs particular care, because it is the only case in which Head itself must change. This is possible only because `insertAtEnd` receives Head by reference. If Head were passed by value, the function would change its own copy, the caller's Head would remain  $\Lambda$ , and New would be lost.

(a) Before:  $\text{Head} = \Lambda$ , so there is no last cell



(b) After:  $\text{Link}(\text{New}) \leftarrow \Lambda$ , then  $\text{Head} \leftarrow \text{New}$



**Figure 4:** Inserting into an empty list. With no last cell to change, New becomes the whole list and Head is set to point to it.

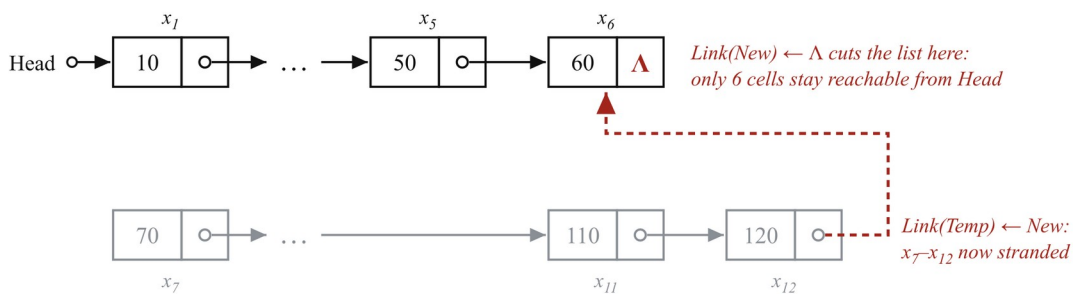
## 5.5 Preserving list integrity

After a successful insertion four properties hold: no cell has been lost, the original order  $x_1$  to  $x_{12}$  is unchanged, exactly one Link is  $\Lambda$ , and the list contains no cycle. Two parts of the algorithm protect these properties.

The first is the assignment  $\text{Link}(\text{New}) \leftarrow \Lambda$ , which protects against a New cell that arrives with a non-null Link, as described in Section 5.3.

The second is the test  $\text{Temp} = \text{New}$ , which protects against New being a cell that is already in the list. Figure 5 shows what would happen without it. If New pointed at  $x_6$ , line 14 would set  $\text{Link}(x_6)$  to  $\Lambda$ , cutting the list after  $x_6$ , and line 15 would then set  $\text{Link}(x_{12})$  to point at  $x_6$ . Head would reach only six cells, and  $x_7$  to  $x_{12}$  could never be reached or released. If New pointed at  $x_{12}$ , the assignment  $\text{Link}(x_{12}) \leftarrow x_{12}$  would create a cycle, and any later traversal would never end. Both outcomes were confirmed by running a copy of the algorithm with the test removed. With the test in place, the algorithm detects that Temp has reached New and returns false without changing anything.

Without the guard, if New pointed at  $x_6$ , a cell already in the list:



**Figure 5:** Why the test  $\text{Temp} = \text{New}$  is needed. Without it, inserting a cell that is already in the list would cut the list short and strand the cells after it.

## 5.6 Tracing the algorithm

Table 6 traces `insertAtEnd` on the twelve-cell list, using the addresses from Table 1.

**Table 6:** Trace of `InsertAtEnd` on the twelve-cell list

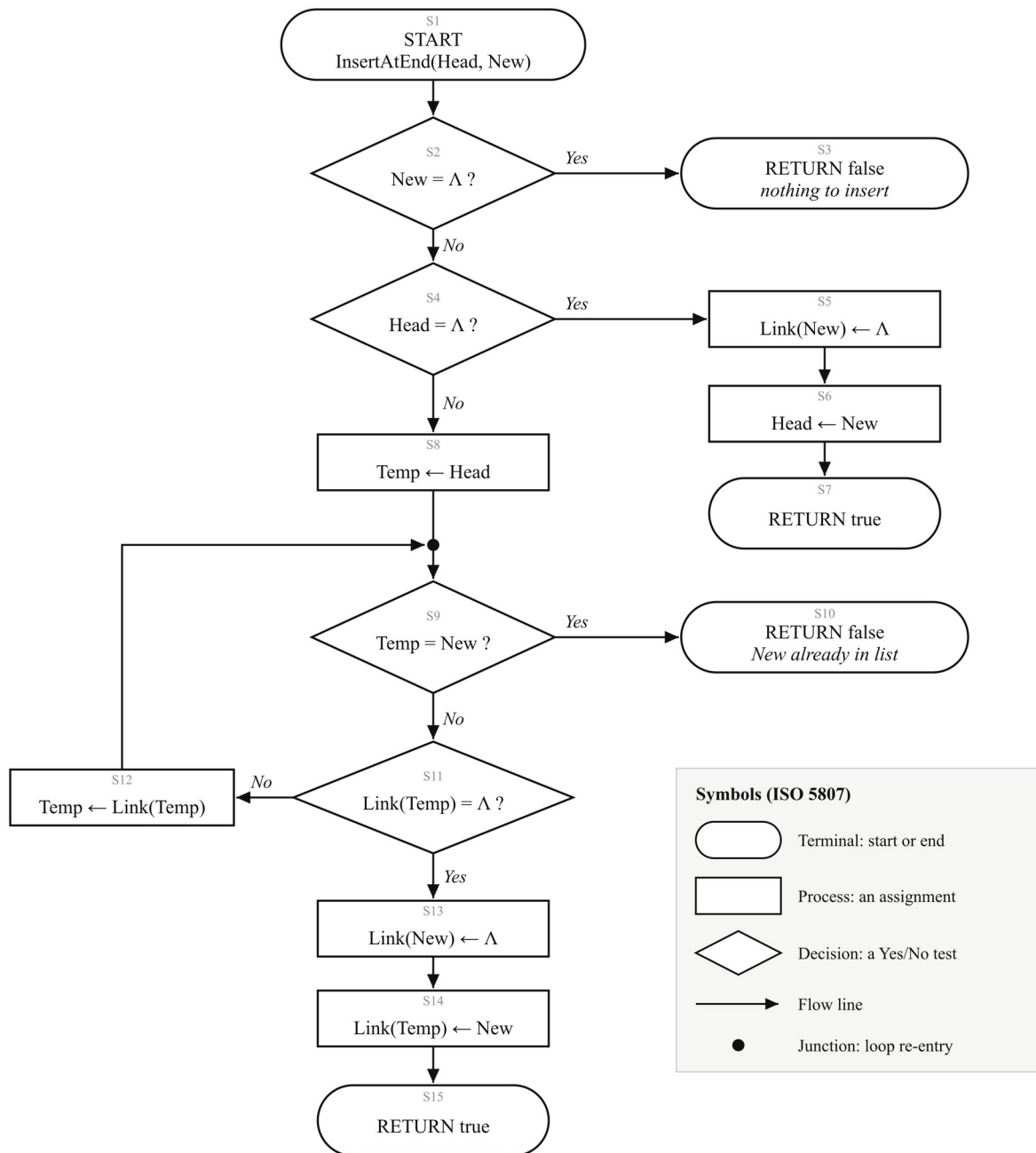
| Pass | Temp                   | Temp = New? | Link(Temp) = $\Lambda$ ? | Action                       |
|------|------------------------|-------------|--------------------------|------------------------------|
| 1    | $x_1$<br>(0x1036f5b70) | No          | No                       | $\text{Temp} \leftarrow x_2$ |
| 2    | $x_2$<br>(0x1036f5900) | No          | No                       | $\text{Temp} \leftarrow x_3$ |

| Pass       | Temp                 | Temp = New? | Link(Temp) = $\Lambda$ ? | Action                                                                   |
|------------|----------------------|-------------|--------------------------|--------------------------------------------------------------------------|
| 3–10       | x3 to x10            | No          | No                       | Temp moves to the next cell                                              |
| 11         | x11<br>(0x1036f59d0) | No          | No                       | Temp $\leftarrow$ x12                                                    |
| 12         | x12<br>(0x1036f59e0) | No          | Yes                      | Exit loop                                                                |
| After loop | x12<br>(0x1036f59e0) |             |                          | Link(New) $\leftarrow \Lambda$ , then Link(x12) $\leftarrow$ 0x1036f59f0 |

Over its twelve passes the loop makes 24 comparisons and moves Temp 11 times, followed by 2 assignments. In general, for a list of  $n$  cells the loop makes  $n$  passes,  $2n$  comparisons and  $n - 1$  moves, which confirms that the running time grows linearly with the length of the list. These counts were checked by instrumenting a copy of the program for lists of 10, 12 and 16 cells.

## 6 Flowchart and Explanation

### 6.1 The flowchart



**Figure 6:** Flowchart of the *InsertAtEnd* algorithm. The main path runs down the centre, the cases that end early branch to the right, and the traversal loop returns along the left to the junction point above *S9*.

Figure 6 presents the insertion algorithm as a flowchart drawn with the symbols defined in ISO 5807 (International Organization for Standardization, 1985). Rounded terminals mark

the start and each possible end; rectangles are processes, here assignments; diamonds are decisions, each with a Yes and a No exit; and arrowed flow lines show the order of execution. A filled junction point marks where the loop returns. Every symbol is labelled S1 to S15 so that it can be matched to the pseudocode and the C++ code in Table 7.

## 6.2 Explanation of each step

The flowchart reads from top to bottom along its main path, with side branches for the cases that finish early.

S1 is the start, with the two inputs Head and New. Decision S2 checks whether New is  $\Lambda$ ; if it is, terminal S3 ends the algorithm with false. Decision S4 checks for an empty list. Its Yes branch runs down the right-hand side: S5 clears Link(New), S6 sets Head to point to New, and S7 ends with true.

On the main path, S8 sets Temp to Head and the flow reaches the junction point. The loop is made up of S9, S11 and S12. Decision S9 checks whether Temp has reached New, and ends at S10 with false if it has. Decision S11 checks whether Temp is on the last cell. If it is not, the No branch leads to S12, which moves Temp to the next cell, and the flow line returns along the left-hand side to the junction and back into S9. The loop appears as a closed path because it is the only part of the algorithm that repeats.

When the answer at S11 is Yes, the loop is finished. S13 clears Link(New), S14 performs the re-link that completes the insertion, and S15 ends with true.

## 6.3 From flowchart to code

Table 7 shows how each flowchart symbol corresponds to a line of pseudocode and a statement in the C++ function.

**Table 7:** Flowchart, pseudocode and C++ correspondence

| Step | Symbol   | Pseudocode line                   | C++ statement                            |
|------|----------|-----------------------------------|------------------------------------------|
| S1   | Terminal | InsertAtEnd(Head, New)            | bool insertAtEnd(Cell*& Head, Cell* New) |
| S2   | Decision | 1: if New = $\Lambda$             | if (New == nullptr)                      |
| S3   | Terminal | 2: return false                   | return false;                            |
| S4   | Decision | 3: if Head = $\Lambda$            | if (Head == nullptr)                     |
| S5   | Process  | 4: Link(New) $\leftarrow \Lambda$ | New->Link = nullptr;                     |
| S6   | Process  | 5: Head $\leftarrow$ New          | Head = New;                              |

| Step     | Symbol    | Pseudocode line                                 | C++ statement                            |
|----------|-----------|-------------------------------------------------|------------------------------------------|
| S7       | Terminal  | 6: return true                                  | return true;                             |
| S8       | Process   | 7: Temp $\leftarrow$ Head                       | Cell* Temp = Head;                       |
| Junction | Connector | 8: loop                                         | while (true) {                           |
| S9       | Decision  | 9: if Temp = New                                | if (Temp == New)                         |
| S10      | Terminal  | 10: return false                                | return false;                            |
| S11      | Decision  | 11–12: if Link(Temp) = $\Lambda$ ,<br>exit loop | if (Temp->Link == nullptr)<br>{ break; } |
| S12      | Process   | 13: Temp $\leftarrow$ Link(Temp)                | Temp = Temp->Link;                       |
| S13      | Process   | 14: Link(New) $\leftarrow$ $\Lambda$            | New->Link = nullptr;                     |
| S14      | Process   | 15: Link(Temp) $\leftarrow$ New                 | Temp->Link = New;                        |
| S15      | Terminal  | 16: return true                                 | return true;                             |

Every symbol in the flowchart corresponds to one line of pseudocode and one statement of C++, and every statement in the function appears in the flowchart. The diagram is therefore a complete and exact picture of the algorithm as implemented, rather than a simplified outline of it.

## 7 Implementation and Testing

The program was compiled with Apple Clang 21 using the C++17 standard and strict warning settings, under which any compiler warning stops the build. Twelve automated tests, containing 63 individual checks, covered list creation, every edge case in Table 5 and repeated insertion. All of the tests passed, as Table 8 shows.

**Table 8:** *Test results*

| Test                                          | Result |
|-----------------------------------------------|--------|
| Creates twelve cells in order                 | Pass   |
| Null-terminates the last cell                 | Pass   |
| Accepts the boundary sizes 10 and 16          | Pass   |
| Rejects sizes outside 10 to 16                | Pass   |
| Appends a cell to the twelve-cell list        | Pass   |
| Empty list: New becomes the Head              | Pass   |
| Inserts into a single-cell list               | Pass   |
| Rejects a null New                            | Pass   |
| Clears a Link that New arrives with           | Pass   |
| Refuses a cell already in the list            | Pass   |
| Repeated insertions keep the list valid       | Pass   |
| deleteList releases the cells and clears Head | Pass   |

Three further checks gave confidence beyond the tests themselves. The tests were run under AddressSanitizer and UndefinedBehaviorSanitizer, which detect invalid memory access and undefined behaviour, and no errors were reported. The macOS leaks tool confirmed that both the tests and the demonstration program finished with no leaked memory, so every cell created with `new` was released with `delete`. Finally, the tests themselves were checked by deliberately introducing seven faults into the algorithm, such as removing the assignment `Link(New) ← A` or re-linking from Head instead of Temp. Six faults were reported as failing tests, and the seventh, removing the check on New, caused an immediate crash, so every fault was detected.

## 8 Conclusion and Self-Assessment

This report created a twelve-cell linked list and an algorithm that inserts a new cell at its end by replacing the  $\Lambda$  in the last cell with the address of New, in  $O(n)$  time and  $O(1)$  extra space. The algorithm handles the empty list and the other edge cases, and it preserves the integrity of the list by clearing  $\text{Link}(\text{New})$  before re-linking and by refusing a cell already in the list. Automated tests, memory-checking tools and deliberate fault injection confirmed that the C++ code behaves exactly as the pseudocode and flowchart describe.

The most useful lesson from the work was that an algorithm which is correct for the normal case can still damage a data structure when it receives unusual input. Working out what would happen if New were already in the list showed that the simplest version of the algorithm loses cells or creates a cycle, a fault that a straightforward demonstration would never reveal. Writing the tests before relying on the results, and then checking that the tests could detect deliberate faults, was what made that kind of fault visible. A natural next step would be to measure the traversal against a tail-pointer version on much longer lists, to quantify the practical cost of the  $O(n)$  search.

## References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge, MA: MIT Press.
- Hoare, C.A.R. (1969) 'An axiomatic basis for computer programming', *Communications of the ACM*, 12(10), pp. 576–580, 583.
- International Organization for Standardization (1985) *ISO 5807:1985 Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts*. Geneva: ISO.
- Knuth, D.E. (1997) *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3rd edn. Reading, MA: Addison-Wesley.
- Levitin, A. (2012) *Introduction to the Design and Analysis of Algorithms*. 3rd edn. Boston, MA: Pearson.
- Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th edn. Upper Saddle River, NJ: Addison-Wesley.
- Stroustrup, B. (2013) *The C++ Programming Language*. 4th edn. Upper Saddle River, NJ: Addison-Wesley.
- Weiss, M.A. (2014) *Data Structures and Algorithm Analysis in C++*. 4th edn. Harlow: Pearson.

## Appendix A: Source Code

The complete C++ implementation, which is also available at [github.com/imranneta5555/DSA](https://github.com/imranneta5555/DSA). It compiles without warnings with the command below, which builds the demonstration program:

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
    main.cpp linked_list.cpp -o demo
```

### A.1 linked\_list.hpp

```
// linked_list.hpp
// A singly linked list of Info-Link cells, following the notation of the
// Unit 3 assignment brief: Head points to the first cell, each cell's Link
// points to the next cell, and the last cell's Link is  $\Lambda$  (nullptr in C++).

#ifndef LINKED_LIST_HPP
#define LINKED_LIST_HPP

#include <cstdint>
#include <ostream>
#include <vector>

// The brief requires the list to be created with between 10 and 16 cells.
constexpr std::size_t MIN_CELLS = 10;
constexpr std::size_t MAX_CELLS = 16;

// One cell of the list. Info holds the data value; Link holds the address of
// the next cell, or nullptr when this cell is the last one.
struct Cell {
    int    Info;
    Cell*  Link;
};

// Allocates one cell on the heap holding `info`, with its Link set to nullptr.
Cell* createCell(int info);

// Builds a list whose cells hold `values` in order and returns its Head.
// Throws std::invalid_argument unless values has MIN_CELLS..MAX_CELLS items.
Cell* createList(const std::vector<int>& values);

// Inserts the cell pointed to by New at the end of the list pointed to by Head.
// Head is passed by reference so an empty list can be given its first cell.
// Returns false and leaves the list untouched if New is null or already in it.
bool insertAtEnd(Cell*& Head, Cell* New);

// Counts the cells reachable from Head.
std::size_t countCells(const Cell* Head);

// Prints the list on one line, e.g. "Head -> [10|0] -> ... -> [120| $\Lambda$ ]".
void printList(std::ostream& out, const Cell* Head);

// Prints one row per cell: position, address, Info and Link.
void printMemoryTable(std::ostream& out, const Cell* Head);

// Releases every cell in the list and sets Head to nullptr.
void deleteList(Cell*& Head);

#endif // LINKED_LIST_HPP
```

### A.2 linked\_list.cpp

```
// linked_list.cpp
// Implementation of the Info-Link linked list and the insert-at-end algorithm.
```

```

#include "linked_list.hpp"

#include <iomanip>
#include <sstream>
#include <stdexcept>
#include <string>

namespace {

// Formats a pointer for display, showing nullptr as the brief's Λ.
std::string addressOf(const Cell* cell) {
    if (cell == nullptr) {
        return "Λ";
    }
    std::ostringstream text;
    text << static_cast<const void*>(cell);
    return text.str();
}

} // namespace

Cell* createCell(int info) {
    return new Cell{info, nullptr};
}

Cell* createList(const std::vector<int>& values) {
    if (values.size() < MIN_CELLS || values.size() > MAX_CELLS) {
        throw std::invalid_argument(
            "a list must have between " + std::to_string(MIN_CELLS) + " and " +
            std::to_string(MAX_CELLS) + " cells, but " +
            std::to_string(values.size()) + " values were given");
    }

    Cell* Head = nullptr; // address of the first cell
    Cell* Tail = nullptr; // address of the last cell built so far

    try {
        for (int value : values) {
            Cell* cell = createCell(value); // new cell, Link already nullptr
            if (Head == nullptr) {
                Head = cell; // first cell: Head points to it
            } else {
                Tail->Link = cell; // previous last cell points to it
            }
            Tail = cell; // it is now the last cell
        }
    } catch (...) {
        deleteList(Head); // allocation failed part-way:
        throw; // release what was built, re-throw
    }

    return Head;
}

bool insertAtEnd(Cell*& Head, Cell* New) {
    // Process 1: there must be a cell to insert.
    if (New == nullptr) {
        return false;
    }

    // Process 2: an empty list has no last cell, so New becomes the whole list.
    if (Head == nullptr) {
        New->Link = nullptr;
        Head = New;
        return true;
    }

    // Process 3: traverse from Head until Temp points to the last cell.
    Cell* Temp = Head;
    while (true) {
        if (Temp == New) {
            return false; // New is already in this list. Re-linking it
        } // would cut the list short and strand the
        // cells after it, or loop the last cell back
        // onto itself, so refuse.
        if (Temp->Link == nullptr) {
            break; // Temp's Link is Λ: Temp is the last cell
        }
    }
}

```

```

    }
    Temp = Temp->Link;        // move Temp on to the next cell
}

// Process 4: re-link. New is made a proper terminal cell first, and only
// then attached, so the list is never left pointing into foreign cells.
New->Link = nullptr;
Temp->Link = New;            // the old  $\Lambda$  is replaced by the address of New
return true;
}

std::size_t countCells(const Cell* Head) {
    std::size_t count = 0;
    for (const Cell* Temp = Head; Temp != nullptr; Temp = Temp->Link) {
        ++count;
    }
    return count;
}

void printList(std::ostream& out, const Cell* Head) {
    out << "Head";
    for (const Cell* Temp = Head; Temp != nullptr; Temp = Temp->Link) {
        out << " -> [" << Temp->Info << '|' << (Temp->Link == nullptr ? " $\Lambda$ " : "o") << ']';
    }
    if (Head == nullptr) {
        out << " ->  $\Lambda$ ";
    }
    out << '\n';
}

void printMemoryTable(std::ostream& out, const Cell* Head) {
    const std::ios_base::fmtflags savedFlags = out.flags(); // restore on exit
    out << std::left
        << " " << std::setw(6) << "Cell" << std::setw(20) << "Address"
        << std::setw(8) << "Info" << "Link\n";

    std::size_t position = 1;
    for (const Cell* Temp = Head; Temp != nullptr; Temp = Temp->Link) {
        out << " " << std::setw(6) << ("x" + std::to_string(position))
            << std::setw(20) << addressOf(Temp)
            << std::setw(8) << Temp->Info
            << addressOf(Temp->Link) << '\n';
        ++position;
    }
    out.flags(savedFlags);
}

void deleteList(Cell*& Head) {
    while (Head != nullptr) {
        Cell* next = Head->Link; // remember the rest of the list first
        delete Head;             // then release this cell
        Head = next;
    }
}

```

## A.3 main.cpp

```

// main.cpp
// Demonstrates the assignment: builds a 12-cell list, inserts a 13th cell at
// the end with insertAtEnd, and prints the list and its memory before and after.

#include "linked_list.hpp"

#include <cstdint>
#include <iostream>
#include <vector>

int main() {
    // Info values x1..x12 = 10, 20, ..., 120.
    std::vector<int> values;
    for (int i = 1; i <= 12; ++i) {
        values.push_back(i * 10);
    }

    std::cout << "Cell layout in memory\n"

```

```

        << "   sizeof(Cell)           = " << sizeof(Cell) << " bytes\n"
        << "   sizeof(Cell*)        = " << sizeof(Cell*) << " bytes\n"
        << "   offsetof(Cell, Info)   = " << offsetof(Cell, Info) << "\n"
        << "   offsetof(Cell, Link)  = " << offsetof(Cell, Link) << "\n\n";

    Cell* Head = createList(values);
    std::cout << "Step 1: list created with " << countCells(Head) << " cells\n";
    printList(std::cout, Head);
    printMemoryTable(std::cout, Head);

    Cell* New = createCell(130); // x13
    std::cout << "\nStep 2: New cell allocated at " << static_cast<const void*>(New)
        << " with Info = " << New->Info << " and Link = \n\n";

    const bool inserted = insertAtEnd(Head, New);
    if (!inserted) {
        delete New; // not taken into the list, so it is still ours to free
    }
    std::cout << "\nStep 3: insertAtEnd returned " << (inserted ? "true" : "false")
        << "; the list now has " << countCells(Head) << " cells\n";
    printList(std::cout, Head);
    printMemoryTable(std::cout, Head);

    deleteList(Head);
    std::cout << "\nStep 4: list deleted; Head is " << (Head == nullptr ? "\n" : "not null")
        << '\n';
    return 0;
}

```

## A.4 test\_linked\_list.cpp

```

// test_linked_list.cpp
// Dependency-free tests for list creation and insertAtEnd.

#include "linked_list.hpp"

#include <cstdint>
#include <functional>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

namespace {

int checksRun = 0;
int checksFailed = 0;

#define CHECK(condition) \
do { \
    ++checksRun; \
    if (!(condition)) { \
        ++checksFailed; \
        std::cerr << "   FAILED line " << __LINE__ << ": " #condition "\n"; \
    } \
} while (false)

std::vector<int> tensUpTo(int count) {
    std::vector<int> values;
    for (int i = 1; i <= count; ++i) {
        values.push_back(i * 10);
    }
    return values;
}

std::vector<int> infoValues(const Cell* Head) {
    std::vector<int> values;
    for (const Cell* Temp = Head; Temp != nullptr; Temp = Temp->Link) {
        values.push_back(Temp->Info);
    }
    return values;
}

Cell* lastCell(Cell* Head) {

```

```

    Cell* Temp = Head;
    while (Temp != nullptr && Temp->Link != nullptr) {
        Temp = Temp->Link;
    }
    return Temp;
}

Cell* cellAt(Cell* Head, std::size_t index) {
    Cell* Temp = Head;
    for (std::size_t i = 0; i < index && Temp != nullptr; ++i) {
        Temp = Temp->Link;
    }
    return Temp;
}

bool contains(const Cell* Head, const Cell* cell) {
    for (const Cell* Temp = Head; Temp != nullptr; Temp = Temp->Link) {
        if (Temp == cell) {
            return true;
        }
    }
    return false;
}

// Floyd's two-pointer check: a list with a cycle never reaches  $\Lambda$ , so a fast
// pointer eventually meets a slow one.
bool hasCycle(const Cell* Head) {
    const Cell* slow = Head;
    const Cell* fast = Head;
    while (fast != nullptr && fast->Link != nullptr) {
        slow = slow->Link;
        fast = fast->Link->Link;
        if (slow == fast) {
            return true;
        }
    }
    return false;
}

// ----- creation

void createList_buildsTwelveCellsInOrder() {
    Cell* Head = createList(tensUpTo(12));
    CHECK(Head != nullptr);
    CHECK(countCells(Head) == 12);
    CHECK(infoValues(Head) == tensUpTo(12));
    CHECK(Head->Info == 10);
    deleteList(Head);
}

void createList_nullTerminatesTheLastCell() {
    Cell* Head = createList(tensUpTo(12));
    Cell* last = lastCell(Head);
    CHECK(last != nullptr);
    CHECK(last->Info == 120);
    CHECK(last->Link == nullptr);
    CHECK(!hasCycle(Head));
    deleteList(Head);
}

void createList_acceptsTheBoundarySizes() {
    Cell* ten = createList(tensUpTo(10));
    Cell* sixteen = createList(tensUpTo(16));
    CHECK(countCells(ten) == 10);
    CHECK(countCells(sixteen) == 16);
    deleteList(ten);
    deleteList(sixteen);
}

void createList_rejectsSizesOutsideTenToSixteen() {
    for (int size : {0, 9, 17}) {
        bool threw = false;
        try {
            Cell* Head = createList(tensUpTo(size));
            deleteList(Head);
        } catch (const std::invalid_argument&) {
            threw = true;
        }
    }
}

```

```

    }
    CHECK(threw);
}
}

// ----- insertAtEnd

void insertAtEnd_appendsToTheTwelveCellList() {
    Cell* Head = createList(tensUpTo(12));
    const Cell* originalHead = Head;
    Cell* New = createCell(130);

    CHECK(insertAtEnd(Head, New));
    CHECK(!hasCycle(Head)); // checked first: safe on any list
    if (hasCycle(Head)) {
        return;
    }
    CHECK(Head == originalHead); // Head is unchanged
    CHECK(countCells(Head) == 13);
    CHECK(lastCell(Head) == New); // New is now the last cell
    CHECK(New->Link == nullptr); // and is null-terminated
    const Cell* oldLast = cellAt(Head, 11); // x12, the old last cell
    CHECK(oldLast != nullptr && oldLast->Link == New);
    CHECK(infoValues(Head) == tensUpTo(13)); // order preserved
    deleteList(Head);
}

void insertAtEnd_intoAnEmptyListMakesNewTheHead() {
    Cell* Head = nullptr;
    Cell* New = createCell(130);

    CHECK(insertAtEnd(Head, New));
    CHECK(Head == New);
    CHECK(countCells(Head) == 1);
    CHECK(Head != nullptr && Head->Link == nullptr);
    if (Head != New) {
        delete New; // not taken into the list
    }
    deleteList(Head);
}

void insertAtEnd_intoASingleCellList() {
    Cell* Head = createCell(10);
    Cell* New = createCell(20);

    CHECK(insertAtEnd(Head, New));
    CHECK(Head->Link == New);
    CHECK(New->Link == nullptr);
    CHECK(countCells(Head) == 2);
    deleteList(Head);
}

void insertAtEnd_rejectsANullNew() {
    Cell* Head = createList(tensUpTo(12));

    CHECK(!insertAtEnd(Head, nullptr));
    CHECK(countCells(Head) == 12);
    CHECK(lastCell(Head)->Link == nullptr);
    deleteList(Head);

    Cell* empty = nullptr;
    CHECK(!insertAtEnd(empty, nullptr));
    CHECK(empty == nullptr);
}

void insertAtEnd_clearsALinkThatNewArrivesWith() {
    Cell* Head = createList(tensUpTo(12));
    Cell* stray = createCell(999);
    Cell* New = createCell(130);
    New->Link = stray; // New arrives pointing at an unrelated cell

    CHECK(insertAtEnd(Head, New));
    CHECK(New->Link == nullptr); // the stray cell is not pulled into the list
    const bool strayAbsorbed = contains(Head, stray);
    CHECK(!strayAbsorbed);
    CHECK(countCells(Head) == 13);
    CHECK(infoValues(Head) == tensUpTo(13));
}

```

```

    deleteList(Head);
    if (!strayAbsorbed) {
        delete stray; // still owned by the caller
    }
}

void insertAtEnd_refusesACellAlreadyInTheList() {
    for (std::size_t index : {std::size_t{0}, std::size_t{5}, std::size_t{11}}) {
        Cell* Head = createList(tensUpTo(12));
        Cell* existing = cellAt(Head, index);

        CHECK(!insertAtEnd(Head, existing)); // head, middle and last cell

        // A cyclic list can't be counted or freed safely, so check that first.
        const bool cyclic = hasCycle(Head);
        CHECK(!cyclic);
        if (cyclic) {
            continue;
        }
        CHECK(countCells(Head) == 12);
        CHECK(infoValues(Head) == tensUpTo(12));
        deleteList(Head);
    }
}

void insertAtEnd_repeatedInsertionsKeepTheListValid() {
    Cell* Head = createList(tensUpTo(10));
    for (int value = 110; value <= 160; value += 10) {
        CHECK(insertAtEnd(Head, createCell(value)));
    }
    CHECK(countCells(Head) == 16);
    CHECK(infoValues(Head) == tensUpTo(16));
    CHECK(lastCell(Head)->Link == nullptr);
    CHECK(!hasCycle(Head));
    deleteList(Head);
}

// ----- deleteList

void deleteList_releasesCellsAndClearsHead() {
    Cell* Head = createList(tensUpTo(12));
    deleteList(Head);
    CHECK(Head == nullptr);

    Cell* empty = nullptr;
    deleteList(empty); // deleting an empty list is safe
    CHECK(empty == nullptr);
}

} // namespace

int main() {
    const std::vector<std::pair<std::string, std::function<void()>>> tests = {
        {"createList builds twelve cells in order",
         createList_buildsTwelveCellsInOrder},
        {"createList null-terminates the last cell",
         createList_nullTerminatesTheLastCell},
        {"createList accepts the boundary sizes 10 and 16",
         createList_acceptsTheBoundarySizes},
        {"createList rejects sizes outside 10-16",
         createList_rejectsSizesOutsideTenToSixteen},
        {"insertAtEnd appends to the twelve-cell list",
         insertAtEnd_appendsToTheTwelveCellList},
        {"insertAtEnd into an empty list makes New the Head",
         insertAtEnd_intoAnEmptyListMakesNewTheHead},
        {"insertAtEnd into a single-cell list",
         insertAtEnd_intoASingleCellList},
        {"insertAtEnd rejects a null New",
         insertAtEnd_rejectsANullNew},
        {"insertAtEnd clears a Link that New arrives with",
         insertAtEnd_clearsALinkThatNewArrivesWith},
        {"insertAtEnd refuses a cell already in the list",
         insertAtEnd_refusesACellAlreadyInTheList},
        {"insertAtEnd repeated insertions keep the list valid",
         insertAtEnd_repeatedInsertionsKeepTheListValid},
        {"deleteList releases cells and clears Head",
         deleteList_releasesCellsAndClearsHead},
    };
}

```

```

};

int testsFailed = 0;
for (const auto& [name, test] : tests) {
    const int failedBefore = checksFailed;
    test();
    const bool passed = checksFailed == failedBefore;
    std::cout << (passed ? " PASS " : " FAIL ") << name << '\n';
    if (!passed) {
        ++testsFailed;
    }
}

std::cout << '\n'
    << tests.size() - static_cast<std::size_t>(testsFailed)
    << " of " << tests.size()
    << " tests passed (" << checksRun - checksFailed << " of " << checksRun
    << " checks)\n";
return testsFailed == 0 ? 0 : 1;
}

```

## Appendix B: Program Output

Output captured from the demonstration and test programs. Memory addresses differ between runs.

### B.1 Demonstration program

```
Cell layout in memory
  sizeof(Cell)      = 16 bytes
  sizeof(Cell*)     = 8 bytes
  offsetof(Cell, Info) = 0
  offsetof(Cell, Link) = 8

Step 1: list created with 12 cells
Head -> [10|o] -> [20|o] -> [30|o] -> [40|o] -> [50|o] -> [60|o] -> [70|o] -> [80|o] -> [90|o]
-> [100|o] -> [110|o] -> [120|Λ]
  Cell  Address      Info  Link
  x1    0x1036f5b70    10    0x1036f5900
  x2    0x1036f5900    20    0x1036f5910
  x3    0x1036f5910    30    0x1036f5960
  x4    0x1036f5960    40    0x1036f5970
  x5    0x1036f5970    50    0x1036f5980
  x6    0x1036f5980    60    0x1036f5990
  x7    0x1036f5990    70    0x1036f59a0
  x8    0x1036f59a0    80    0x1036f59b0
  x9    0x1036f59b0    90    0x1036f59c0
  x10   0x1036f59c0   100    0x1036f59d0
  x11   0x1036f59d0   110    0x1036f59e0
  x12   0x1036f59e0   120    Λ

Step 2: New cell allocated at 0x1036f59f0 with Info = 130 and Link = Λ

Step 3: insertAtEnd returned true; the list now has 13 cells
Head -> [10|o] -> [20|o] -> [30|o] -> [40|o] -> [50|o] -> [60|o] -> [70|o] -> [80|o] -> [90|o]
-> [100|o] -> [110|o] -> [120|o] -> [130|Λ]
  Cell  Address      Info  Link
  x1    0x1036f5b70    10    0x1036f5900
  x2    0x1036f5900    20    0x1036f5910
  x3    0x1036f5910    30    0x1036f5960
  x4    0x1036f5960    40    0x1036f5970
  x5    0x1036f5970    50    0x1036f5980
  x6    0x1036f5980    60    0x1036f5990
  x7    0x1036f5990    70    0x1036f59a0
  x8    0x1036f59a0    80    0x1036f59b0
  x9    0x1036f59b0    90    0x1036f59c0
  x10   0x1036f59c0   100    0x1036f59d0
  x11   0x1036f59d0   110    0x1036f59e0
  x12   0x1036f59e0   120    0x1036f59f0
  x13   0x1036f59f0   130    Λ

Step 4: list deleted; Head is Λ
```

### B.2 Test program

```
PASS  createList builds twelve cells in order
PASS  createList null-terminates the last cell
PASS  createList accepts the boundary sizes 10 and 16
PASS  createList rejects sizes outside 10-16
PASS  insertAtEnd appends to the twelve-cell list
PASS  insertAtEnd into an empty list makes New the Head
PASS  insertAtEnd into a single-cell list
PASS  insertAtEnd rejects a null New
PASS  insertAtEnd clears a Link that New arrives with
PASS  insertAtEnd refuses a cell already in the list
PASS  insertAtEnd repeated insertions keep the list valid
PASS  deleteList releases cells and clears Head

12 of 12 tests passed (63 of 63 checks)
```